

Internship Report

EINETCORP

February 2025 – May 2025

1 Introduction

This report presents the summary of my four-month internship at Einetcorp, a company focused on building assistive AI-based technologies for the visually challenged. The internship duration was from February 2025 to May 2025. As an AI and ML Engineer Intern, I worked on a real-time object detection system integrated into smart glasses. These glasses identify objects in the surroundings and communicate them to the user through voice output, significantly helping the visually impaired navigate more independently.

This report outlines my contributions, learning outcomes, tools and technologies used, and the impact of the project within the team and the company.

2 Company Profile

Company Name: Einetcorp

Location: KLE Technological University, CITIE bldg. Karnataka, Hubli

Year of Establishment: March 2021

Domain Expertise: Computer Vision, Artificial Intelligence, Edge Computing, Assistive Technologies

Einetcorp is a growing technology firm that develops AI-powered wearables aimed at assisting visually challenged users. Its flagship product—AI-enabled smart glasses—employs real-time object detection, edge inference, and text-to-speech systems to provide audio feedback about the surrounding environment.

3 Objectives of the Internship

The objectives set at the beginning of the internship were as follows:

- To understand real-world software and AI development workflows in a product-focused tech company.
- To gain hands-on experience with object detection models such as YOLOv5, along with OpenCV, Streamlit, and Text-to-Speech Engines.
- To contribute to the development of a real-time object detection application integrated with smart glasses.
- To build a user interface that allows media upload, webcam streaming, and generates downloadable PDF/CSV reports.
- To enhance the system's ability to speak out detected objects, improving accessibility.
- To participate in debugging, optimization, and deployment planning.

4 Overview of Machine Learning

Machine Learning (ML) plays a foundational role in AI-driven systems like object detection. During the internship, I revisited the core concepts of ML and gained hands-on experience in real-world implementation.

Key Topics Covered

- **Supervised Learning:** Training the model with labeled data. Applied for object classification.
- **Unsupervised Learning:** Grouping and clustering data without labels (explored theoretically).
- **Algorithms Studied:** Support Vector Machines (SVM), K-Nearest Neighbors (KNN), Decision Trees, Logistic Regression, etc.
- **Model Lifecycle:**
 - Data Collection and Preprocessing

- Model Training and Validation
- Accuracy Evaluation (Confusion Matrix, F1 Score)
- Real-time Deployment of trained model
- **Python Libraries Used:**
 - Pandas and NumPy for data handling
 - Scikit-learn for model building and evaluation
 - Matplotlib and Seaborn for data visualization

5 Tasks and Projects Undertaken

Project Title: Real-Time Object Detection System Using Computer Vision

A. Frontend and Backend Tasks

- Built a user-friendly interface using Streamlit for image/video upload and live webcam streaming.
- Designed real-time result display with bounding boxes and audio predictions using pytsx3.

B. Data Preprocessing and Model Training

- Integrated pretrained YOLOv5 models for object detection.
- Applied real-time frame capture using OpenCV.
- Processed outputs to extract labels and confidence scores.

C. API Integration and Output Management

- Stored detection results in a CSV log.
- Generated PDF reports using the reportlab library.
- Provided real-time audio output for accessibility.

D. Deployment

- Project hosted and tested locally via Streamlit.
- Prepared version for deployment on cloud platforms (Heroku, AWS EC2).

E. Tools and Technologies Used

- Git/GitHub for version control and collaboration
- Postman for API testing and debugging
- MongoDB Atlas (optional) for database storage (future scope)
- Python, YOLOv5, OpenCV, Streamlit, ReportLab

6 Skills Acquired

Technical Skills

- Hands-on experience with the MERN Stack including React.js and Node.js.
- Developed and deployed Python-based ML models using YOLOv5, OpenCV, Streamlit.
- Worked with REST APIs, JWT authentication, and real-time object detection pipelines.
- Understood data preprocessing, model evaluation, and report generation.

Tools and Software Used

- VS Code, Git, GitHub, Postman, MongoDB Atlas
- Jupyter Notebook for ML experimentation

Soft Skills

- Improved collaboration via regular team reviews.

- Enhanced communication through documentation and presentations.
- Managed time effectively to meet development deadlines.

7 Challenges Faced

Technical Challenges

- Integrating webcam feed into Streamlit without lag.
- Debugging API response formatting.
- Managing data imbalance in object detection.

Adaptation Challenges

- Learning reportlab and pyttsx3 within a tight timeline.
- Handling full-stack responsibilities across frontend, backend, and ML components.

Resolution Strategies

- Consulted mentors and used team Slack for guidance.
- Used online documentation and Stack Overflow.
- Employed time-blocking for task management.

8 Conclusion

The internship at Einetcorp was a deeply educational and professionally enriching experience. I had the opportunity to work on a live AI product that aids the visually challenged, contributing directly to a socially impactful mission.

I learned to apply academic concepts of Computer Vision and Machine Learning to real-world applications. Working with YOLOv5, OpenCV, and Streamlit gave me a full-stack view of intelligent system development.

The collaborative environment, exposure to deployment, and technical mentoring helped me grow as an AI/ML engineer. This internship has strengthened my career path in AI-powered assistive technologies.

9 Appendix

Project Snapshots

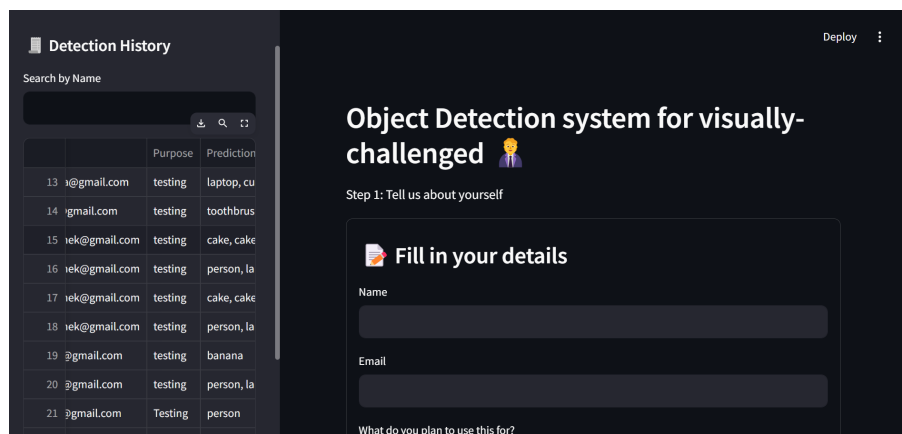


Figure 1: Detection history section

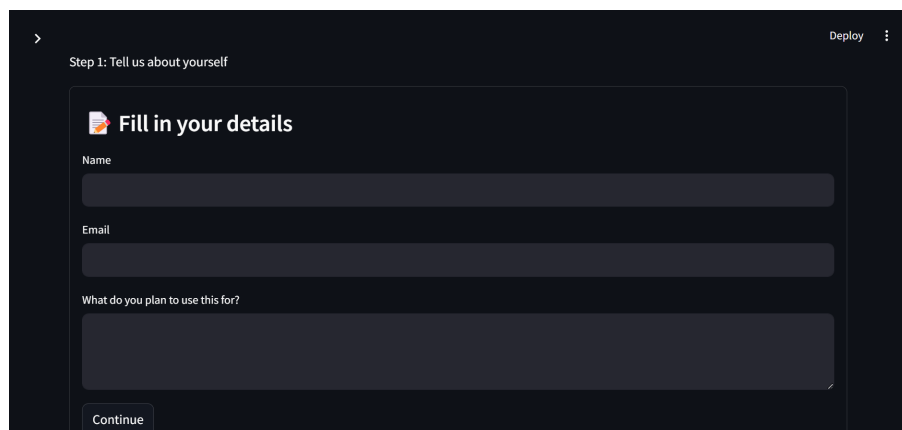


Figure 2: Filling in user details

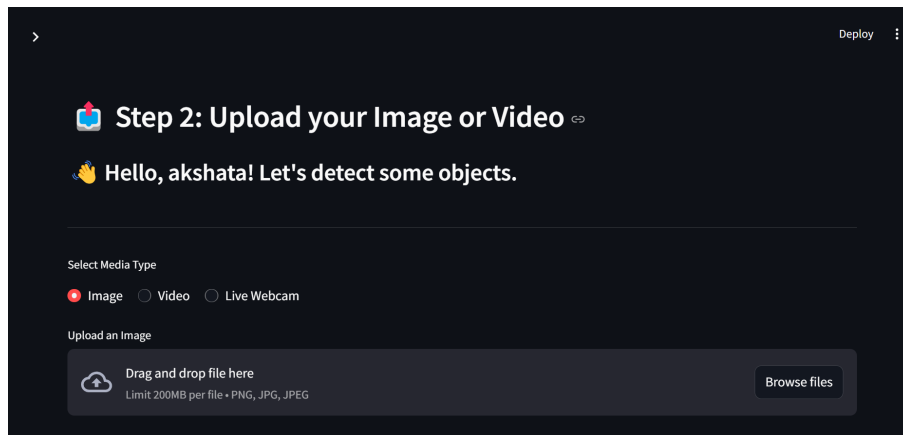


Figure 3: Upload pictures or videos or live webcam

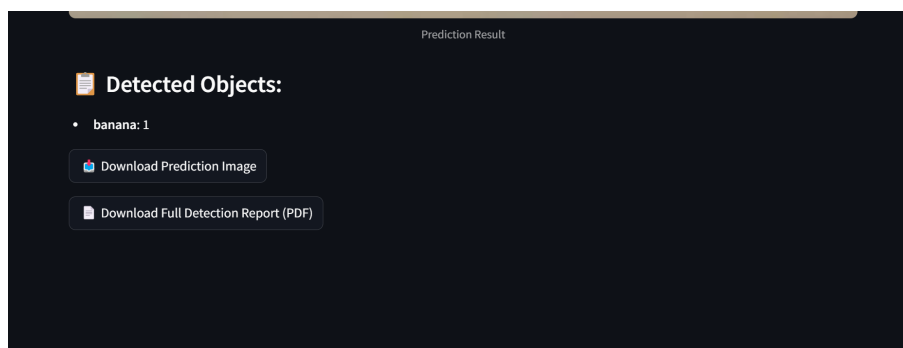


Figure 4: Detected objects in pictures

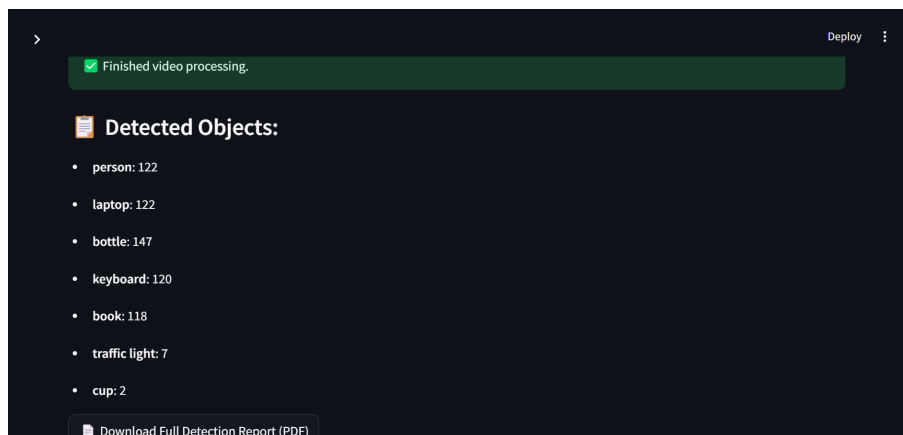


Figure 5: Detected objects list from video

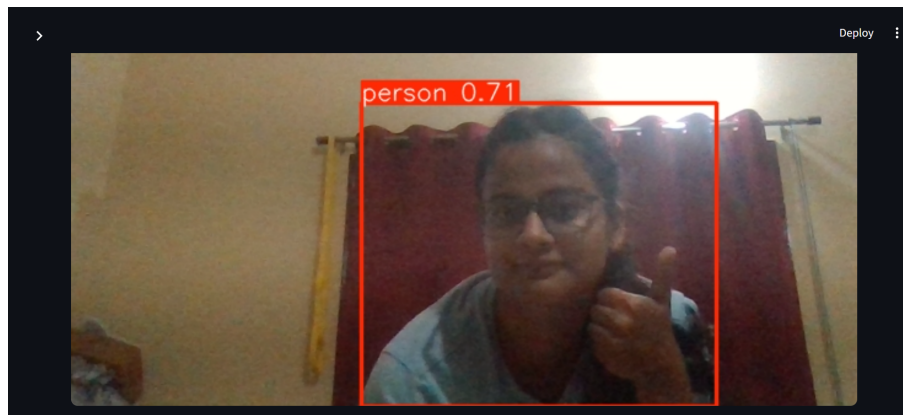


Figure 6: Live webcam

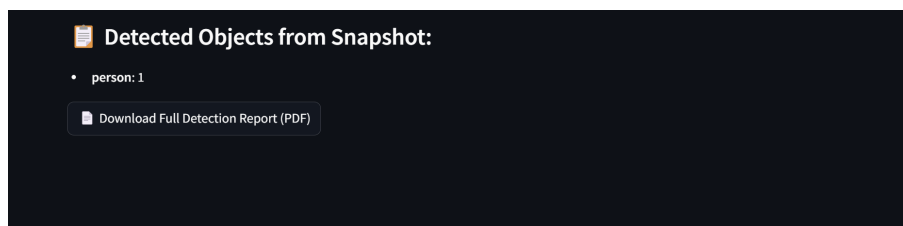


Figure 7: Detected objects from Live webcam

Appendix B: Python Code Used

```
import streamlit as st
import torch
from PIL import Image
import tempfile
import os
import cv2
import time
import pandas as pd
from collections import Counter
from datetime import datetime
import pyttsx3
from reportlab.lib.pagesizes import letter
from reportlab.pdfgen import canvas
from reportlab.lib.utils import ImageReader
import numpy as np
from io import BytesIO

st.set_page_config(page_title="Smart Object Detector", layout="wide"
)

@st.cache_resource
def load_model():
    return torch.hub.load('ultralytics/yolov5', 'yolov5s',
        pretrained=True)

model = load_model()

def speak_predictions(predictions):
    engine = pyttsx3.init()
```

```
engine.setProperty('rate', 150)

if predictions:
    unique_objects = sorted(set(predictions))
    spoken_text = "Detected objects are: " + ", ".join(
        unique_objects)
else:
    spoken_text = "No objects detected."

engine.say(spoken_text)
engine.runAndWait()

def save_to_csv(name, email, purpose, predictions):
    df = pd.DataFrame({
        "Name": [name],
        "Email": [email],
        "Purpose": [purpose],
        "Predictions": [", ".join(predictions)],
        "Timestamp": [datetime.now().strftime("%Y-%m-%d %H:%M:%S")]
    })
    if os.path.exists("user_predictions.csv"):
        df.to_csv("user_predictions.csv", mode='a', header=False,
            index=False)
    else:
        df.to_csv("user_predictions.csv", index=False)

def delete_row_by_index(index):
    df = pd.read_csv("user_predictions.csv")
    df.drop(index=index, inplace=True)
    df.to_csv("user_predictions.csv", index=False)

def generate_pdf_report(name, email, purpose, predictions,
    image_path=None):
```

```
pdf_path = "detection_report.pdf"
c = canvas.Canvas(pdf_path, pagesize=letter)
width, height = letter
c.setFont("Helvetica-Bold", 16)
c.drawString(50, height - 50, "Smart Object Detection Report")
c.setFont("Helvetica", 12)
c.drawString(50, height - 80, f"Name: {name}")
c.drawString(50, height - 100, f"Email: {email}")
c.drawString(50, height - 120, f"Purpose: {purpose}")
c.drawString(50, height - 140, f"Timestamp: {datetime.now().
    strftime('%Y-%m-%d %H:%M:%S')}")
c.setFont("Helvetica-Bold", 14)
c.drawString(50, height - 170, "Detected Objects:")
c.setFont("Helvetica", 12)
if predictions:
    counts = Counter(predictions)
    y = height - 190
    for cls, count in counts.items():
        c.drawString(70, y, f"- {cls}: {count}")
        y -= 20
else:
    c.drawString(70, height - 190, "No objects detected.")
if image_path and os.path.exists(image_path):
    try:
        img = Image.open(image_path)
        img.thumbnail((300, 300))
        img.save("resized_preview.jpg")
        c.drawImage(ImageReader("resized_preview.jpg"), 300,
            height - 400, width=200, preserveAspectRatio=True)
    except:
```

```

        c.drawString(50, height - 250, "Preview image could not
            be loaded.")

    c.save()

    return pdf_path

# Detection History
st.sidebar.markdown("##          Detection History")
if os.path.exists("user_predictions.csv"):
    df_history = pd.read_csv("user_predictions.csv")
    search_name = st.sidebar.text_input("Search by Name")
    if search_name:
        df_history = df_history[df_history["Name"].str.contains(
            search_name, case=False)]
    for i, row in df_history.tail(10).iterrows():
        st.sidebar.markdown(f"**{row['Name']}** - {row['Predictions
            ']} ")
        st.sidebar.write(f"{row['Timestamp']}")
        if st.sidebar.button("          ", key=f"delete_{i}"):
            delete_row_by_index(i)
            st.experimental_rerun()
else:
    st.sidebar.write("No detection history yet.")

if "form_submitted" not in st.session_state:
    st.session_state.form_submitted = False

if not st.session_state.form_submitted:
    st.markdown("## Object Detection System for Visually-Challenged
        \nStep 1: Tell us about yourself")
    with st.form("user_info_form"):
        st.markdown("###          Fill in your details")

```

```
name = st.text_input("Name")
email = st.text_input("Email")
purpose = st.text_area("What do you plan to use this for?",
    max_chars=150)
submitted = st.form_submit_button("Continue")

if submitted:
    st.session_state.form_submitted = True
    st.session_state.name = name
    st.session_state.email = email
    st.session_state.purpose = purpose
    st.rerun()

if st.session_state.form_submitted:
    st.markdown(f"##          Step 2: Upload your Image, Video, or Use
    Webcam")
    st.markdown(f"###          Hello, **{st.session_state.name}**! Let
    's detect some objects.")
    media_type = st.radio("Select Media Type", ["Image", "Video", "
    Live Webcam"], horizontal=True)

    if media_type == "Image":
        image_file = st.file_uploader("Upload an Image", type=["png"
            , "jpg", "jpeg"])
        if image_file:
            image = Image.open(image_file).convert("RGB")
            st.image(image, caption="Uploaded Image",
                use_container_width=True)
            with st.spinner("Predicting..."):
                results = model(image)
                predictions = results.pandas().xyxy[0]['name'].
                    tolist()
```

```
results.render()

pred_img = Image.fromarray(results.ims[0])

st.image(pred_img, caption="Prediction Result",
         use_container_width=True)

st.markdown("###      Detected Objects:")

if predictions:
    for cls, count in Counter(predictions).items():
        st.write(f"- **{cls}**: {count}")
else:
    st.write("No objects detected.")

speak_predictions(predictions)

img_path = "prediction_result.jpg"
pred_img.save(img_path)
with open(img_path, "rb") as file:
    st.download_button("      Download Prediction Image",
                       file, file_name="prediction_result.jpg")

save_to_csv(st.session_state.name, st.session_state.
            email, st.session_state.purpose, predictions)
pdf_path = generate_pdf_report(
    st.session_state.name,
    st.session_state.email,
    st.session_state.purpose,
    predictions,
    img_path
)

with open(pdf_path, "rb") as f:
```

```
        st.download_button("          Download Full Detection  
        Report (PDF)", f, file_name="detection_report.pdf  
        ")

elif media_type == "Video":
    video_file = st.file_uploader("Upload a Video", type=["mp4",  
        "mov", "avi"])
    if video_file:
        tfile = tempfile.NamedTemporaryFile(delete=False)
        tfile.write(video_file.read())
        st.video(tfile.name)
        stframe = st.empty()
        cap = cv2.VideoCapture(tfile.name)
        all_preds = []
        while cap.isOpened():
            ret, frame = cap.read()
            if not ret:
                break
            results = model(frame)
            predictions = results.pandas().xyxy[0]['name'].  
                tolist()
            all_preds.extend(predictions)
            results.render()
            rendered_frame = results.ims[0]
            stframe.image(rendered_frame, channels="BGR",  
                use_container_width=True)
            time.sleep(0.03)
        cap.release()
        st.markdown("###          Detected Objects:")
        if all_preds:
            for cls, count in Counter(all_preds).items():
```

```
        st.write(f"- **{cls}**: {count}")
    else:
        st.write("No objects detected.")

    speak_predictions(all_preds)

    save_to_csv(st.session_state.name, st.session_state.
                email, st.session_state.purpose, all_preds)
    pdf_path = generate_pdf_report(
        st.session_state.name,
        st.session_state.email,
        st.session_state.purpose,
        all_preds
    )
    with open(pdf_path, "rb") as f:
        st.download_button("          Download Full Detection
                            Report (PDF)", f, file_name="detection_report.pdf
                            ")

elif media_type == "Live Webcam":
    st.markdown("###          Live Webcam")
    st.warning("Webcam will run for 10 seconds. Then snapshot
               will be auto-captured.")
    snapshot_img = None
    all_preds = []
    cap = cv2.VideoCapture(0)
    start_time = time.time()
    stframe = st.empty()
    while time.time() - start_time < 10:
        ret, frame = cap.read()
        if not ret:
```



```
        st.error("Failed to access webcam.")
        break

    results = model(frame)
    predictions = results.pandas().xyxy[0]['name'].tolist()
    all_preds.extend(predictions)
    results.render()

    rendered = results.ims[0]
    stframe.image(rendered, channels="BGR",
                  use_container_width=True)
cap.release()

st.success("Snapshot taken after 10 seconds")
snapshot_img = rendered
st.image(snapshot_img, caption="Snapshot with Detections",
         use_container_width=True)

st.markdown("###      Detected Objects:")
if all_preds:
    for cls, count in Counter(all_preds).items():
        st.write(f"- **{cls}**: {count}")
else:
    st.write("No objects detected.")

speak_predictions(all_preds)
img_path = "webcam_snapshot.jpg"
cv2.imwrite(img_path, snapshot_img)
with open(img_path, "rb") as file:
    st.download_button("      Download Snapshot Image",
                      file, file_name="webcam_snapshot.jpg")
```

```
save_to_csv(st.session_state.name, st.session_state.email,
            st.session_state.purpose, all_preds)
pdf_path = generate_pdf_report(
    st.session_state.name,
    st.session_state.email,
    st.session_state.purpose,
    all_preds,
    img_path
)
with open(pdf_path, "rb") as f:
    st.download_button("Download Full Detection
                        Report (PDF)", f, file_name="detection_report.pdf")
else:
    st.info("Please fill out the form above to get started.")
```

Listing 1: Streamlit + YOLOv5 Object Detection Script